

3.1 Array Design

Sue was again enlisted by her mother to help her make sense of some stock data. While it is easy to determine the starting price of the stock (the first item on the list) or the ending price of the stock (the last item on the list), it is much more tedious to find the high and low values. Rather than laboriously search the list by hand, Sue writes a program to find these values.

Objectives

By the end of this class, you will be able to:

- Search for a value in an array.
- Look up a value in an array.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare an array to solve a problem (Chapter 3.0).
- Write a loop to traverse an array (Chapter 3.0).
- Predict the output of a code fragment containing an array (Chapter 3.0).
- Pass an array to a function (Chapter 3.0).

Overview

Arrays are used for a wide variety of tasks. The most common usage is when a collection of homogeneous data is needed. Common examples include text (arrays of characters), lists of numbers (a set of grades), and lists of more complex things (lists of addresses or students for example). In each case, individual members of the list can be referenced by index. Problems involving this usage of arrays typically involve filling, manipulating, and extracting data from a list.

Another use for an array would be to look up a value from a list. Consider selecting an item off a menu or looking up the price on an order sheet. In both of these cases, equivalent logic can be written with IF/ELSE statements. Storing the data in a table, however, often takes less memory, requires less code, and is much easier to update. Problems involving this usage of arrays typically involve looking up data in tables.

Lists of Data

When solving problems involving lists of data, it is common to need to write a loop to visit every element of the list. Most of these problems can be reduced to the following two questions:

1. How do you iterate through the list (usually using a standard FOR loop)?
2. What happens to each item on the list?

To illustrate this principle, three examples will be used: filling a list from a file of data, finding an item from a list of unsorted data, and finding an item from a list of sorted data.

Example 3.1 – Read From File

Demo

This example will demonstrate how to fill an array from a list of numbers in a file. This is a common function to write: fill an array from a given file name, an array to be filled, and the number of items in the array.

Problem

Fill the array data with the values in the following file:

```
1 3 6 2 9 3
```

Solution

In order to write a function to read this data into an array, it is necessary to answer the question “what needs to happen to each item in the list?” The answer is: read it from the file (using `fin >>`) and save it in the array (using `fout <<`). To accomplish this, our function needs to take three parameters: `fileName` or the location from which we will be reading the data, `data` or where we will be placing the data, and `max` or the size or capacity of the array data.

Observe how we need to send some information back to the caller, namely how many items we successfully read. This is most conveniently done through the return type where 0 indicates a failure. Consider the following function:

```
int readFile(const char fileName[],    // use const because it will not change
             int data[],              // the output of the function
             int max)                 // capacity of data, it will not change
{
    // open the file for reading
    ifstream fin(fileName);           // open the input stream to fileName
    if (fin.fail())                   // never forget the error checking
    {
        cout << "ERROR: Unable to read file "
              << fileName << endl;    // display the filename we tried to read
        return 0;                     // return the error condition: none read
    }

    // loop through the file, reading the elements one at a time
    int numRead = 0;                  // initially none were read
    while (numRead < max &&           // don't read more than the list holds
           fin >> data[numRead])     // read and check for errors
        numRead++;                   // increment the index by one

    // close the file and return
    fin.close();                      // never forget to close the file
    return numRead;                   // report the number successfully read
}
```

Observe how we make sure to check that we are not putting more items in the list than there is room. If the list holds 10 but the file has 100 items, we should still only read 10.

Challenge

We did not traverse the array using the standard FOR loop even though all three parts (initialization, condition, and increment) are present. As a challenge, try to modify the above function so a FOR loop is used to read the data from the file instead of a WHILE loop. Which solution is best?

See Also

The complete solution is available at [3-1-readFile.cpp](#) or:

```
/home/cs124/examples/3-1-readFile.cpp
```



Example 3.1 – Searching an Unsorted List

Demo	Another common array problem is to find a given item in an unsorted list. In this case, the ordering of the list is completely random (as unsorted lists are) so it is necessary to visit every item in the list.
Problem	Write a function to determine if a given search value is present in a list of integers: <pre>bool linearSearch(const int numbers[], int size, int search);</pre> If the value <code>search</code> is present in <code>numbers</code>, return <code>true</code>, otherwise, return <code>false</code>.
Solution	The first step to solving this problem is to answer the question “what needs to happen to each item in the list?” The answer is: compare it against the sought-after item. This will be accomplished by iterating through the array of numbers, comparing each entry against the search value. <pre>bool linearSearch(const int numbers[], // the list to be searched int size, // how many items are in the list int search) // the term being searched for { // walk through every element in the list for (int i = 0; i < size; i++) // standard FOR loop for an array if (search == numbers[i]) // compare each against the search item return true; // if found, then leave with true // not found if we reached the end return false; }</pre> Observe how the larger the list (<code>size</code>), the longer it will take. We call this a “linear search” because the cost of the search is directly proportional to the size of the list.
Challenge	Finding if an item exists in a list is essentially the same problem as finding the largest (or smallest) item in a list. As a challenge, modify the above function to return the largest number: <pre>int findLargestValue(const int numbers[], int size);</pre> To accomplish this, declare a variable that contains the largest value currently found. Each item is compared against this value. If the largest number currently found is smaller than the current item being compared, then update the value with the current item. After every item in the list has been compared, the value of the largest is returned.
See Also	The complete solution is available at 3-1-linearSearch.cpp or: <pre>/home/cs124/examples/3-1-linearSearch.cpp</pre> 

Example 3.1 – Searching a Sorted List

Demo

It turns out that people rarely perform linear searches. Imagine how long it would take to look up a word in the dictionary! This example will demonstrate how to do a binary search.

Problem

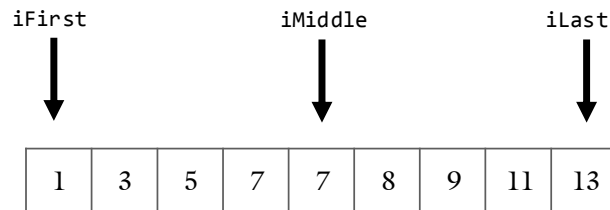
Write a function to determine if a given search value is present in a list of integers:

```
bool binarySearch(const int numbers[], int size, int search);
```

If the value `search` is present in `numbers`, return `true`, otherwise, return `false`.

The binary search algorithm works much like searching for a hymn in the hymnal:

1. Start in the middle (`iMiddle`) by opening the hymnal to the center of the book.
2. If the hymn number is greater, then you can rule out the first half of the book. Thus the first possible page (`iFirst`) it could be on is the middle (`iMiddle`), the last is the end (`iLast`).
3. If the hymn number is smaller then you can rule out the second half of the book.
4. Repeat steps 1-3. With each iteration, we either find the hymn or rule out half of the remaining pages. Thus `iFirst` and `iLast` get closer and closer together. If `iFirst` and `iLast` are the same, then our hymn is not present and we quit the search.



Observe how the question “what needs to happen to each item in the list?” is answered with “decide if we should focus on the top half or bottom half of the list.”

```
bool binarySearch(const int numbers[], int size, int search)
{
    int iFirst = 0;                // iFirst and iLast represent the range
    int iLast = size - 1;          // of possible values: the whole list

    while (iLast >= iFirst)        // as long as the range is not empty
    {
        int iMiddle = (iLast + iFirst) / 2; // find the center (step (1) above)

        if (numbers[iMiddle] == search)    // if we found it, then stop
            return true;
        else if (numbers[iMiddle] > search) // if middle is bigger, focus on the
            iLast = iMiddle - 1;           // beginning of the list (step (2))
        else                               // otherwise (smaller), focus on the
            iFirst = iMiddle + 1;          // end of the list (step (3))
        // continue (step (4))
    }

    // only got here if we didn't find it
    return false;                    // failure
}
```

See Also

The complete solution is available at 3-1-binarySearch.cpp or:

```
/home/cs124/examples/3-1-binarySearch.cpp
```



Table Lookup

Arrays are also a very useful tool in solving problems involving looking up data from a table or a list of values. This class of problems is typically solved in two steps:

1. Create a table of the data to be referenced.
2. Write code to extract the data from the table.

This is best illustrated with an example. Consider the following code to convert a number grade into a letter grade:

```
/******  
 * COMPUTE LETTER GRADE  
 * Compute the letter grade from the  
 * passed number grade  
 *****/  
char computeLetterGrade(int numberGrade)  
{  
    assert(numberGrade >= 0 && numberGrade <= 100);  
  
    // table to be referenced  
    char grades[] =  
    { //0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%  
      'F', 'F', 'F', 'F', 'F', 'F', 'D', 'C', 'B', 'A', 'A'  
    };  
  
    assert(numberGrade / 10 >= 0);  
    assert(numberGrade / 10 < sizeof(grades) / sizeof(grades[0]));  
    return grades[numberGrade / 10];    // Divide will give us the 10's digit  
}
```

When using this technique, it is important to spend extra time and thought on the representation of the data in the table. The goal is to represent the data as clearly (read: error-free) as possible and to make it as easy to extract the data as possible. This programming technique is called **data-driven design**.

Observe how we do not have a FOR loop to iterate through the list. Since we were careful about how the list was ordered (where the index of the `grades` array correspond to the first 10's digit of the `numberGrade` array), we can look up the letter grade directly.

Finally, while it may seem excessive to have three asserts in a function containing only two statements, these asserts go a long way to find bugs and prevent unpredictable behavior.

Example 3.1 – Tax Bracket

Demo

This example will demonstrate a table-loopup design for arrays. In this case, the tax table will be put in a series of arrays.

Problem

Consider the following tax table:

If taxable income is over--	But not over--	The tax is:
\$0	\$15,100	10% of the amount over \$0
\$15,100	\$61,300	\$1,510.00 plus 15% of the amount over 15,100
\$61,300	\$123,700	\$8,440.00 plus 25% of the amount over 61,300
\$123,700	\$188,450	\$24,040.00 plus 28% of the amount over 123,700
\$188,450	\$336,550	\$42,170.00 plus 33% of the amount over 188,450
\$336,550	no limit	\$91,043.00 plus 35% of the amount over 336,550

Compute a user's tax bracket based on his income.

Solution

The first part of the solution is to create three arrays representing the lower part of the tax bracket, the upper part of the tax bracket, and the taxation rate. The second part is to loop through the brackets, seeing if the user's income falls within the upper and lower bounds. If it does, the corresponding tax rate is returned.

```
int computeTaxBracket(int income)
{
    int lowerRange[] =           // the 1st column of the tax table
    { // 10%   15%   25%   28%   33%   35%
      0, 15100, 61300, 123700, 188450, 336550
    };
    int upperRange[] =           // the 2nd column
    { // 10%   15%   25%   28%   33%   35%
      15100, 61300, 123700, 188450, 339550, 999999999
    };
    int bracket[]                // the bracket
    {
        10,    15,    25,    28,    33,    35
    };

    for (int i = 0; i < 6; i++)    // the index for the three arrays
        if (lowerRange[i] <= income && income <= upperRange[i])
            return bracket[i];

    return -1;                    // not in range (negative income?)!
}
```

Challenge

As a challenge, modify this function to compute the actual income. This will require a fourth array: the fixed amount. See if you can put your function in Project 1 and get it to pass testBed.

See Also

The complete solution is available at [3-1-computeTaxBracket.cpp](#) or:

/home/cs124/examples/3-1-computeTaxBracket.cpp



Problem 1

What is the output of the following code?

```
{
    int a[4];

    for (int i = 0; i < 4; i++)
        a[i] = i;

    for (int j = 3; j >= 0; j--)
        cout << a[j];

    cout << endl;
}
```

Answer:

Please see page 218 for a hint.

Problem 2

What is the output of the following code?

```
{
    char a[] = {'t', 'm', 'q'};
    char b[] = {'a', 'z', 'b'};
    char c[3];

    for (int i = 0; i < 3; i++)
        if (a[i] > b[i])
            c[i] = a[i];
        else
            c[i] = b[i];

    for (int i = 0; i < 3; i++)
        cout << c[i];

    cout << endl;
}
```

Answer:

Please see page 237 for a hint.

Problem 3

Complete the code to count the number of even and odd numbers:

```
void displayEvenOdd(const int values[],
                    int num)
{
    //determine even/odd
    int numEvenOdd[2] = {0, 0};

    // display
    cout << "Number even: "
          << numEvenOdd[0] << endl;
    cout << "Number odd: "
          << numEvenOdd[1] << endl;
}
```

Please see page 237 for a hint.

Problem 4

Fibonacci is a sequence of numbers where each number is the sum of the previous two:

$$F(n) := \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

--	--	--	--	--	--	--	--	--	--

Write the code to complete the Fibonacci sequence and store the results in an array.

```
void fibonacci(int array[], int num)
{

}

}
```

Please see page 237 for a hint.

Assignment 3.1

Start with Assignment 3.0 and modify the function `averageGrades()` so that it does not take into account grades with the value -1. In this case, -1 indicates the assignment was not completed yet so it should not factor in the average.

Examples

Two examples... The user input is underlined.

Example 1

```
Grade 1: 90  
Grade 2: 86  
Grade 3: 95  
Grade 4: 76  
Grade 5: 92  
Grade 6: 83  
Grade 7: 100  
Grade 8: 87  
Grade 9: 91  
Grade 10: -1  
Average Grade: 88%
```

Notice how the -1 for the 10th grade is not factored into the average.

Example 2

```
Grade 1: -1  
Grade 2: -1  
Grade 3: -1  
Grade 4: -1  
Grade 5: -1  
Grade 6: -1  
Grade 7: -1  
Grade 8: -1  
Grade 9: -1  
Grade 10: -1  
Average Grade: ---%
```

Since all of the grades are -1, there is no average. You will need to handle this condition.

Assignment

The test bed is available at:

```
testBed cs124/assign31 assignment31.cpp
```

Don't forget to submit your assignment with the name "Assignment 31" in the header.

Please see page 235 for a hint.